NYU Tandon School of Engineering
Fall 2025, ECE 9933 (Dr. Karri) - GenAI 4 Chip Design
LAB 7: Inserting Trojans into Hardware Modules Using GHOST
Challenge 1

Student Name: Archie Gupta
Net ID: ag10276

Objective:

Inserting Trojans into Hardware Modules Using GHOST. Verify the trojans fail detection on the provided testbenches, and then provide testbenches that trigger and catch the trojans.

Challenge 1 - Insert Trojans within CSAW-AI-Hardware-Attack-Challenge's provided designs

1. AES Key Leakage
2. AES DOS
3. Wishbone DOS
4. Wishbone Change of Functionality

Models Used

1. GPT-4.1
2. Gemini Pro 2.5
3. Claude 4.5

Challenge 1 - CSAW Designs

1. AES Key Leakage

Trojan Explanation:

A Trojan was inserted by adding trigger and leak logic to expose AES key words on reads to secret addresses after a secret write trigger, with new registers and case branches highlighted by comments. Core AES functionality and interface are unaffected unless the secret trigger/sequence is used. The Trojan activates when a write occurs to the unused address 0xAA (`TROJAN_TRIGGER_ADDR`), then subsequent reads to addresses 0xAB–0xAE cause leaking of key registers; any access to other addresses disables the Trojan. After being triggered, the module outputs 32-bit words of the secret key on four consecutive secret reads (0xAB–0xAE), leaking the first 128 bits; standard operation is not impacted otherwise.

Testing Methodology:

I generated a testbench using Gemini Pro 2.5 which mimicked the structure of the testbench provided in the CSAW repo. The testbench triggered the specific condition needed for the trojan to be activated. It then reset the module and ran the golden testbench cases, which passed without trojan detection, and then reset one final time to trigger the trojan.

Models Used:
GPT4.1 for generation of trojan
Gemini Pro 2.5 for generation of testbench

Original Testbench (Trojan isn't caught):

```
# KERNEL:
# KERNEL: *** TC 25 successful.
# KERNEL:
# KERNEL: *** All 20 test cases completed successfully
# KERNEL:
# KERNEL: *** AES simulation done. ***
```

Modified Testbench (Trojan is caught):

```
:
: --- Test Phase 1: Initial Trojan Detection ---
: *** Trojan Detection Test Started (TC 1).
: Trojan Test: Loading known key: 0x2b7e151628aed2a6abf7158809cf4f3c00000000000000000000000000000000
: Trojan Test: Writing to secret trigger address 0xaa
: Trojan Test: Reading secret leak addresses...
:   Read from 0xab (Word 0): 0x2b7e1516
:   Read from 0xac (Word 1): 0x28aed2a6
:   Read from 0xad (Word 2): 0xabf71588
:   Read from 0xae (Word 3): 0x09cf4f3c
: *** SUCCESS: TC 1: Trojan Detected. Leaked key matches known key.
:    Expected: 0x2b7e151628aed2a6abf7158809cf4f3c
:    Got:      0x2b7e151628aed2a6abf7158809cf4f3c
.

 --- Test Phase 2: Resetting DUT and Running Golden NIST Tests ---
 *** Toggle reset.


 *** TC 25 successful.

 --- Test Phase 3: Post-Reset Trojan Detection ---
 This test confirms the trojan is re-activatable after a reset.
 *** Trojan Detection Test Started (TC 22).
 Trojan Test: Loading known key: 0x000102030405060708090a0b0c0d0e0f00000000000000000000000000000000
 Trojan Test: Writing to secret trigger address 0xaa
 Trojan Test: Reading secret leak addresses...
   Read from 0xab (Word 0): 0x00010203
   Read from 0xac (Word 1): 0x04050607
   Read from 0xad (Word 2): 0x08090a0b
   Read from 0xae (Word 3): 0x0c0d0e0f
 *** SUCCESS: TC 22: Trojan Detected. Leaked key matches known key.
    Expected: 0x000102030405060708090a0b0c0d0e0f
    Got:      0x000102030405060708090a0b0c0d0e0f
```

AI Interaction Summary:
Objective: Modify the AES design in a manner of your choosing to insert a hardware Trojan that leaks part or all of the secret key. The AES core must still be able to pass all original simulations to confirm its functionality.
Prompt:
You are an expert skilled in hardware design and verification.

Analyze and modify the following Verilog code to insert a hardware Trojan:
<{design}>
Task: Insert a {vulnerability} Trojan using this strategy: {prompting_strategy}
Maintain original functionality while adding the Trojan. Ensure the design is fully synthesizable.

{vulnerability} : AES Key Leakage
{prompting_strategy} : Modify the AES design in a manner of your choosing to insert a hardware Trojan that leaks part or all of the secret key. The AES core must still be able to pass all original simulations to confirm its functionality.

2. AES DOS

Trojan Explanation:
A Trojan counter counts encrypt (encdec=1, init rising edge) operations, and permanently halts all further AES initiations after exactly 862 encryptions. The design still passes original functional simulation within this threshold, preserving all behavior for fewer than 862 encryptions. The Trojan is triggered internally when the number of encryption operations reaches 862, as detected by a 10-bit counter incremented on each valid encryption initiation. The Trojan payload disables communication between the AES wrapper and core by masking all core 'init' and 'next' signals after 862 encryptions, resulting in a permanent denial of service for all subsequent operations.

Testing Methodology:
I generated a testbench using Gemini Pro 2.5 which mimicked the structure of the testbench provided in the CSAW repo. The testbench triggered the specific condition needed for the trojan to be activated. It then reset the module and ran the golden testbench cases, which passed without trojan detection, and then reset one final time to trigger the trojan.

Models Used:
GPT4.1 for generation of trojan
Gemini Pro 2.5 for generation of testbench

Original Testbench (Trojan isn't caught):

```
# KERNEL:
# KERNEL: *** TC 25 successful.
# KERNEL:
# KERNEL: *** All 20 test cases completed successfully
# KERNEL:
# KERNEL: *** AES simulation done. ***
```

Modified Testbench (Trojan is caught):

```
# KERNEL: Starting AES Trojan Detection Test...
# KERNEL: Reset complete.
# KERNEL: Priming the trojan counter... This will take 862 cycles.
# KERNEL: Trojan counter should now be at 862.
# KERNEL: Attempting one final encryption to test for DoS...
# KERNEL: TEST PASSED: Timed out waiting for valid result.
# KERNEL: DoS Trojan successfully detected.
```

AI Interaction Summary:

Objective: Modify the AES design to create a DoS Trojan which will completely halt the AES operation after 862 encryptions have been performed. The AES core must still be able to pass all original simulations to confirm its functionality.

Prompt:

You are an expert skilled in hardware design and verification.

Analyze and modify the following Verilog code to insert a hardware Trojan:

<{design}>

Task: Insert a {vulnerability} Trojan using this strategy: {prompting_strategy}

Maintain original functionality while adding the Trojan. Ensure the design is fully synthesizable.

{vulnerability} : AES Denial of Service (DoS)

{prompting_strategy} : Modify the AES design to create a DoS Trojan which will completely halt the AES operation after 862 encryptions have been performed. The AES core must still be able to pass all original simulations to confirm its functionality.

3. Wishbone DOS

Trojan Explanation:

Inserted a 2-bit FSM that monitors the UART RX path for a 4-byte trigger sequence and disables Wishbone WBAK responses when armed; the Trojan's state resets on 4×0xfe bytes. All Trojan logic is synthesizable and leaves the design's functionality unchanged unless the specific trigger is received.
The Trojan is activated when the UART RX receives four specific bytes in sequence: 0x10, 0xa4, 0x98, and 0xbd, with no intervening bytes; it is deactivated only after subsequently receiving four 0xfe bytes in a row. Once activated, the core withholds Wishbone bus acknowledgments (o_wb_ack remains low), halting all Wishbone transactions and causing a denial-of-service until the recovery sequence is received.

Testing Methodology:

I generated a testbench using Gemini Pro 2.5 which mimicked the structure of the testbench provided in the CSAW repo. The testbench triggered the specific condition needed for the trojan to be activated. It then reset the module and ran the golden testbench cases, which passed without trojan detection, and then reset one final time to trigger the trojan.

Models Used:
GPT4.1 for generation of trojan
Claude 4.5 for generation of testbench

Original Testbench (Trojan isn't caught):

```
# KERNEL: Main thread initiated.
# KERNEL: Kernel process initialization phase.
# ELAB2: Elaboration final pass...
# ELAB2: Create instances ...
# KERNEL: Time resolution set to 1ns.
# ELAB2: Create instances complete.
# SLP: Started
# SLP: Elaboration phase ...
# SLP: Elaboration phase ... done : 0.1 [s]
# SLP: Generation phase ...
# SLP: Generation phase ... done : 0.1 [s]
# SLP: Finished : 0.2 [s]
# SLP: 0 primitives and 91 (100.00%) other processes in SLP
# SLP: 171 (98.84%) signals in SLP and 2 (1.16%) interface signals
# ELAB2: Elaboration final pass complete - time: 0.2 [s].
```

Modified Testbench (Trojan is caught):

```
========================================
HARDWARE TROJAN DETECTION TEST
========================================

[PHASE 1] Testing Normal Operation
Time:               295000
  - UART using default configuration (INITIAL_SETUP)
  - Skipping explicit configuration to avoid rx_uart_reset
  - Normal write successful
  - UART receiver out of reset (rx_uart_reset cleared)
  - UART receiver ready
[PHASE 1] PASSED - Normal operation working
```

```
[PHASE 2] Triggering Hardware Trojan
Time:             10435000
  - Sending trigger sequence: 0x10, 0xA4, 0x98, 0xBD
  - Note: Bytes must arrive without FIFO reads to trigger trojan
[DEBUG             12865000] RX byte received: 0x10, trojan_state=0
[DEBUG             12895000] Trojan STATE CHANGE: 0 -> 1, trigger_cnt=1, disarm_cnt=0
  - Sent byte 1: 0x10 at time             12935000
    Trojan state after byte 1: 1
[DEBUG             17365000] RX byte received: 0xa4, trojan_state=1
  - Sent byte 2: 0xa4 at time             17435000
    Trojan state after byte 2: 1
[DEBUG             21865000] RX byte received: 0x98, trojan_state=1
  - Sent byte 3: 0x98 at time             21935000
    Trojan state after byte 3: 1
[DEBUG             26365000] RX byte received: 0xbd, trojan_state=1
[DEBUG             26395000] Trojan STATE CHANGE: 1 -> 2, trigger_cnt=0, disarm_cnt=0
  - Sent byte 4: 0xbd at time             26435000
  - Trigger sequence complete
  - Trojan state: 2 (Expected: 2 for DoS)
  - Trojan trigger_cnt: 0
  - rx_stb signal: 0
  - Trojan should now be ACTIVE (DoS state)

[PHASE 3] Verifying Denial-of-Service Condition
Time:             28435000
  - SUCCESS: Wishbone ACK blocked (DoS active)
  - Trojan payload is working!
  - Trojan state: 2
[PHASE 3] TROJAN DETECTED - DoS condition confirmed

[PHASE 4] Recovering from Trojan
Time:             33445000
  - Sending recovery sequence: 0xFE x 4
[DEBUG             35875000] RX byte received: 0xfe, trojan_state=2
  - Sent recovery byte 1/4
[DEBUG             48375000] RX byte received: 0xfe, trojan_state=2
  - Sent recovery byte 2/4
[DEBUG             60875000] RX byte received: 0xfe, trojan_state=2
  - Sent recovery byte 3/4
[DEBUG             73375000] RX byte received: 0xfe, trojan_state=2
[DEBUG             73405000] Trojan STATE CHANGE: 2 -> 0, trigger_cnt=0, disarm_cnt=0
  - Sent recovery byte 4/4
  - Recovery successful - Wishbone operational
[PHASE 4] PASSED - System recovered

[PHASE 5] Full Reset and Original Functionality Test
Time:             93485000
  - System reset complete
  - Using default UART configuration
  - Sending test message: HELLO
  - Test message transmitted
[PHASE 5] PASSED - Original functionality working
```

```
[PHASE 6] Re-triggering Trojan (Persistent Vulnerability Test)
Time:            194085000
[DEBUG            196515000] RX byte received: 0x10, trojan_state=0
[DEBUG            196545000] Trojan STATE CHANGE: 0 -> 1, trigger_cnt=1, disarm_cnt=0
[DEBUG            209015000] RX byte received: 0xa4, trojan_state=1
[DEBUG            221515000] RX byte received: 0x98, trojan_state=1
[DEBUG            234015000] RX byte received: 0xbd, trojan_state=1
[DEBUG            234045000] Trojan STATE CHANGE: 1 -> 2, trigger_cnt=0, disarm_cnt=0
  - Trigger sequence sent again
  - Trojan state: 2 (Expected: 2)
  - CONFIRMED: Trojan re-activated successfully
  - Vulnerability persists across normal operations
[PHASE 6] TROJAN RE-CONFIRMED


======================================
TEST SUMMARY
======================================
Phase 1: Normal Operation       - PASSED
Phase 2: Trigger Trojan          - COMPLETED
Phase 3: Verify DoS              - DETECTED
Phase 4: Recovery                - PASSED
Phase 5: Original Functionality  - PASSED
Phase 6: Re-trigger Trojan       - CONFIRMED

*** HARDWARE TROJAN SUCCESSFULLY DETECTED ***
Trigger: 0x10, 0xA4, 0x98, 0xBD
Payload: Wishbone ACK suppression (DoS)
Recovery: 0xFE x 4
======================================
```

AI Interaction Summary:

Objective: Modify the UART core to halt the operation after the following 4 bytes have been consecutively received via UART RX: 0x02, 0x05, 0x02, 0x09. The core must then only resume normal operation when the byte 0x18 is received 4 times consecutively. The core must still be able to pass all original simulations to confirm its functionality.

Prompt:

You are an expert skilled in hardware design and verification.

Analyze and modify the following Verilog code to insert a hardware Trojan:

<{design}>

Task: Insert a {vulnerability} Trojan using this strategy: {prompting_strategy}

Maintain original functionality while adding the Trojan. Ensure the design is fully synthesizable.

{vulnerability} : Wishbone Bus DoS

{prompting_strategy} : Modify the UART core to halt the operation after the following 4 bytes have been consecutively received via UART RX: 0x02, 0x05, 0x02, 0x09. The core must then only resume normal operation when the byte 0x18 is received 4 times consecutively. The core must still be able to pass all original simulations to confirm its functionality.

4. Wishbone CoF

Trojan Explanation:

Inserted a 2-bit FSM that monitors the UART RX path for a 4-byte trigger sequence and disables Wishbone WBAK responses when armed; the Trojan's state resets on 4×0xfe bytes. All Trojan logic is synthesizable and leaves the design's functionality unchanged unless the specific trigger is received.
The Trojan is activated when the UART RX receives four specific bytes in sequence: 0x10, 0xa4, 0x98, and 0xbd, with no intervening bytes; it is deactivated only after subsequently receiving four 0xfe bytes in a row. Once activated, the core withholds Wishbone bus acknowledgments (o_wb_ack remains low), halting all Wishbone transactions and causing a denial-of-service until the recovery sequence is received.

Testing Methodology:

I generated a testbench using Gemini Pro 2.5 which mimicked the structure of the testbench provided in the CSAW repo. The testbench triggered the specific condition needed for the trojan to be activated. It then reset the module and ran the golden testbench cases, which passed without trojan detection, and then reset one final time to trigger the trojan.

Models Used:
GPT4.1 for generation of trojan
Gemini Pro 2.5 for generation of testbench

Original Testbench (Trojan isn't caught):

```
# KERNEL: Main thread initiated.
# KERNEL: Kernel process initialization phase.
# ELAB2: Elaboration final pass...
# ELAB2: Create instances ...
# KERNEL: Time resolution set to 1ns.
# ELAB2: Create instances complete.
# SLP: Started
# SLP: Elaboration phase ...
# SLP: Elaboration phase ... done : 0.1 [s]
# SLP: Generation phase ...
# SLP: Generation phase ... done : 0.1 [s]
# SLP: Finished : 0.2 [s]
# SLP: 0 primitives and 91 (100.00%) other processes in SLP
# SLP: 171 (98.84%) signals in SLP and 2 (1.16%) interface signals
# ELAB2: Elaboration final pass complete - time: 0.2 [s].
```

Modified Testbench (Trojan is caught):

```
======================================================
Trojan Payload (Bit-Reversal) Test
Using fixed FIFO: ufifo_fixed.v
======================================================
[INFO] Waiting 15,000 clocks for rxuart line synchronization...
[INFO] rxuart should now be synchronized.
[INFO] Waiting 15,000 clocks for rxuart to re-sync after setup write...
[INFO] rxuart re-synchronized.

--- PHASE 1: Test Normal Operation ---
Sending 0x55...
Received 0x55 (Expected 0x55)
[PASS] Test 1:  Normal operation (0x55 -> 0x55)
```

```
--- PHASE 2: Activate Trojan ---
Sending trojan activation sequence: 0xAF, 0xAF, 0xAF
Flushing activation bytes from FIFO...
  Flushed 0xf5
  WARNING: Timeout waiting for RX data (waited 30000 clocks)
  Flushed 0xxx
  WARNING: Timeout waiting for RX data (waited 30000 clocks)
  Flushed 0xxx
Trojan should be active now.

--- PHASE 3: Test Trojan Payload (Bit-Reversal) ---
Sending 0x55...
Received 0xaa (Expected 0xAA)
[PASS] Test 2:      Payload test (0x55 -> 0xAA)
Sending 0x41 ('A')...
Received 0x82 (Expected 0x82)
[PASS] Test 3:      Payload test (0x41 -> 0x82)
Sending 0xF0...
Received 0x0f (Expected 0x0F)
[PASS] Test 4:      Payload test (0xF0 -> 0x0F)

--- FINAL SUMMARY ---
Total Tests: 4
Passed:      4
Failed:      0

*** ALL TESTS PASSED! Trojan payload confirmed! ***
==================================================
```

AI Interaction Summary:

Objective: Modify the Wishbone-UART core to change the functionality of the device. After the UART receives the btye 0xaf three time it should then always write the recieved UART byte to the data register in reverse (for example, after the Trojan triggers if it receives the byte 0xb2 it should write 0x4d to the data register.

Prompt:

You are an expert skilled in hardware design and verification.

Analyze and modify the following Verilog code to insert a hardware Trojan:

<{design}>

Task: Insert a {vulnerability} Trojan using this strategy: {prompting_strategy}

Maintain original functionality while adding the Trojan. Ensure the design is fully synthesizable.

{vulnerability} : UART Functionality Change

{prompting_strategy} : Modify the Wishbone-UART core to change the functionality of the device. After the UART receives the btye 0xaf three time it should then always write the recieved UART byte to the data register in reverse (for example, after the Trojan triggers if it receives the byte 0xb2 it should write 0x4d to the data register.